

Lab Practice: Building a Simple ELT Pipeline with Python, BigQuery, and Apache Airflow

Overview

You're going to build a small, real ELT (Extract, Load, Transform) pipeline using four genuinely messy CSV files: customer records, orders, products, and promotions, from the same coffee chain example used throughout this module. Your job is to load that raw, dirty data into a cloud data warehouse (BigQuery) exactly as-is, then clean it and join it into one analysis-ready table sitting in that same warehouse, and have Apache Airflow run the whole thing automatically.

By the end of this lab, you'll have touched every major topic from this module in one working example: a cloud data warehouse, the ELT technique (including why loading dirty data first is the point, not a mistake), building an analysis-ready table, and Airflow orchestration.

Learning objectives

By completing this lab, you will be able to:

- Set up a Google Cloud project and use BigQuery without needing a credit card
- Load raw data into a cloud warehouse using Python (the "L" in ELT)
- Clean and combine raw warehouse data into one analysis-ready table (the "T" in ELT)
- Build an Airflow DAG that orchestrates both steps in the correct order
- Explain, in your own words, why this pipeline is ELT and not ETL

Part 0: Set up BigQuery (no credit card needed)

BigQuery has a free "sandbox" mode that doesn't require a billing account or credit card, so you can complete this entire lab at no cost. It gives you 1 TB of query processing and 10 GB of storage per month, more than enough for this lab. Note one limitation you should know about upfront: sandbox mode doesn't support INSERT/UPDATE/DELETE statements or streaming inserts, only batch loading and SELECT-based queries, which is exactly what this lab uses anyway.

1. Go to console.cloud.google.com and sign in with a Google account.
2. Create a new project. Name it something like `coffee-chain-dw`. Note the **Project ID** shown, you'll need it later (it may not be exactly the name you typed).
3. In the search bar, go to **BigQuery**. It will activate in sandbox mode automatically since you haven't added billing.
4. In the BigQuery console, create two datasets inside your project (click your project name, then "Create Dataset"):

- raw (this is where the untouched, raw data will land)
 - analytics (this is where the cleaned, joined table will live)
5. Set up a way for Python to talk to BigQuery. The simplest method for this lab is a service account key:
 - Go to **IAM & Admin > Service Accounts > Create Service Account**.
 - Name it something like `airflow-bq-access`.
 - Grant it the **BigQuery Admin** role (simplest for learning purposes; in a real job, you'd grant a narrower role).
 - After creating it, click into it, go to the **Keys** tab, **Add Key > Create New Key > JSON**. This downloads a `.json` file, keep it safe, it's essentially a password.

Part 1: Set up Airflow with the Astro CLI

Use the Astro CLI method from the lesson, since it's the fastest path to a working Airflow environment.

1. Install Docker Desktop if you don't already have it, and make sure it's running.
2. Install the Astro CLI (see the lesson for the command for your OS: `brew install astro` on macOS, `winget install -e --id Astronomer.Astro` on Windows, or the `curl` command on Linux).
3. Create your project:

```
mkdir coffee_elt_lab && cd coffee_elt_lab
astro dev init
```

4. Add the Python packages this lab needs. Open the `requirements.txt` file Astro created and add:

```
google-cloud-bigquery
pandas
pyarrow
```

5. Place five files into the `include/` folder (this folder is exactly for "files your DAGs need," per the lesson):
 - Your downloaded service account JSON key, renamed to `gcp_key.json`.
 - The four data files: `customers_dirty.csv`, `orders_dirty.csv`, `products_dirty.csv`, `promotions_dirty.csv` (described below).
6. Start Airflow:

```
astro dev start
```

The first time it starts, follow whatever login instructions appear in your terminal.

The data: four real, messy tables

Instead of clean sample data, you're working with four CSV files that look like what actually comes out of a real production database: `customers_dirty.csv`, `orders_dirty.csv`, `products_dirty.csv`, and `promotions_dirty.csv`. Place all four in your project's `include/` folder.

This is deliberate. Loading data exactly as it is, dirty, before cleaning it, is precisely what ELT means. If this were ETL, you'd be expected to clean this before it ever reached the warehouse. Here, you load it broken, and fix it only after it's already inside BigQuery.

Before you start coding, open each file and actually look through it. Don't take "clean sample data" for granted here, part of this lab is finding the real data quality problems yourself: inconsistent formatting, missing values, and mismatches between tables, and figuring out what your transform step needs to handle before you can fix any of it.

Part 2: Build the pipeline

Create a new file in your `dags/` folder called `coffee_elt_pipeline.py`. Build a DAG named `coffee_elt_pipeline` with two tasks, connected so transform runs after `extract_and_load` completes:

1. **extract_and_load**: reads all four CSV files from the `include/` folder and loads each one, untouched, into its own table under a raw dataset in BigQuery (`raw.customers`, `raw.orders`, `raw.products`, `raw.promotions`). Running this task more than once should not create duplicate rows.
2. **transform**: builds `analytics.orders_enriched` from `raw.orders`, joined to `raw.customers` and `raw.promotions`. How you clean and join the data is up to you, whether that's a SQL query run inside BigQuery, a pandas-based approach, or a mix of both, as long as the final result ends up saved as a real table in BigQuery. Your approach doesn't need to fix every data quality issue in every column, just handle what's actually needed to produce the columns below correctly. Make sure no order gets silently dropped just because something about it doesn't cleanly match another table.

`analytics.orders_enriched` should end up with one row per order and these columns:

Column	What it should contain
<code>order_id</code>	The order's ID, unchanged from <code>raw.orders</code>
<code>customer_id</code>	The customer's ID, unchanged from <code>raw.orders</code>
<code>customer_name</code>	The customer's first and last name combined into one clean value
<code>order_date_clean</code>	The order date as a real DATE, correctly parsed regardless of which raw format it came in as

Column	What it should contain
<code>status_clean</code>	The order status, standardized down to a small, consistent set of values
<code>promo_code_clean</code>	The promo code, cleaned up so it actually matches the promotions table
<code>discount_pct</code>	The discount percentage from the matching promotion, if there is one
<code>promo_channel</code>	The promotion's channel, if there is one
<code>total_amount</code>	The order total, unchanged from <code>raw.orders</code>

One hint:

- Inside your DAG code, files you placed in `include/` aren't referenced by that relative name, Astro CLI maps that folder to `/usr/local/airflow/include/` inside the running container. So your service account key path would be `/usr/local/airflow/include/gcp_key.json`, and similarly for each CSV file.

Part 3: Run it and verify

1. Open the Airflow UI (<http://localhost:8080>) and manually trigger the `coffee_elt_pipeline` DAG.
2. Confirm both tasks turn green. If something fails, click into the task and read the log, it will usually tell you exactly what went wrong (a common one: forgetting to call `.result()`, which means your code moves on before BigQuery has actually finished the job).
3. Go to the BigQuery console and confirm:
 - All four raw tables exist under the raw dataset (`raw.customers`, `raw.orders`, `raw.products`, `raw.promotions`), each with the same row count as its source CSV, and still messy, exactly as it came in.
 - `analytics.orders_enriched` exists, has one row per order (should be 103 rows, matching `raw.orders`), and its `status_clean` column only contains the 7 clean values (Completed, Pending, Cancelled, Refunded, Shipped, In Progress, Unknown), not the original 17.
4. Trigger the DAG a second time, then re-check the row counts from step 3. They should be identical to the first run, not doubled, which confirms your `write_disposition` is actually working.